CODE 1:

```c
#include <stdio.h>
#include <unistd.h>

int main()
{
   pid_t pid;
   pid = fork(); // Create a new process
   if (pid < 0)
   {
      printf("Fork failed!\n");
   }
   else if (pid == 0)
   {
      printf("This is the child process.\n");
      printf("Child PID: %d\n", getpid());
   }
   else
   {
      printf("This is the parent process.\n");
      printf("Parent PID: %d, Child PID: %d\n", getpid(), pid);
   }
return 0;
}
```

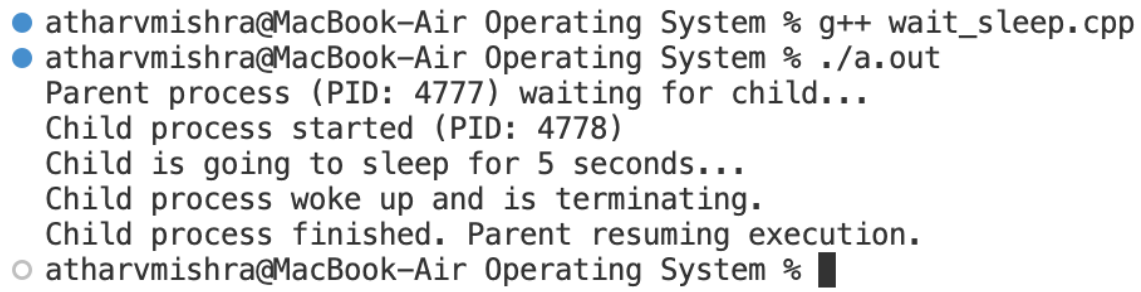
OUTPUT:

```
atharvmishra@MacBook-Air Operating System % g++ fork.cpp
atharvmishra@MacBook-Air Operating System % ./a.out
This is the parent process.
Parent PID: 5060, Child PID: 5063
This is the child process.
Child PID: 5063
atharvmishra@MacBook-Air Operating System %
```

CODE 2:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
int main() {
pid_t pid;
pid = fork();
if (pid < 0) {
printf("Fork failed!\n");
}else if (pid == 0) {
printf("Child process started (PID: %d)\n", getpid());
printf("Child is going to sleep for 5 seconds...\n");
sleep(5);
printf("Child process woke up and is terminating.\n");
}else {
printf("Parent process (PID: %d) waiting for child...\n", getpid());
wait(NULL);
printf("Child process finished. Parent resuming execution.\n");
}
return 0;
}
```

OUTPUT:



CODE 3:

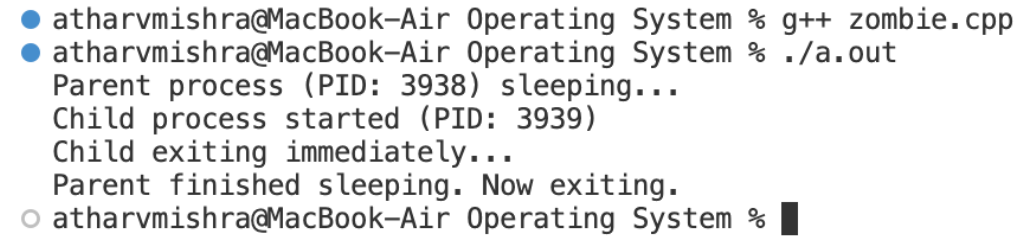
```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
int main() {
   pid_t pid = fork();
   if (pid < 0) {
      printf("Fork failed!\n");
   }
   else if (pid == 0) {
      printf("Child process started (PID: %d, Parent PID: %d)\n", getpid(), getppid());
      sleep(5);
      printf("After sleep, Child PID: %d, New Parent PID: %d (Orphan adopted by init/systemd)\n",
           getpid(), getppid());
   }
   else {
      printf("Parent process exiting (PID: %d)\n", getpid());
   }
   return 0;
}
```

OUTPUT (ORPHAN PROCESS):

```
atharvmishra@MacBook-Air Operating System % g++ orphan.cpp
atharvmishra@MacBook-Air Operating System % ./a.out
Parent process exiting (PID: 4386)
Child process started (PID: 4387, Parent PID: 4386)
atharvmishra@MacBook-Air Operating System % After sleep, Child PID: 4387, New Parent PID: 1 (Orphan adopted by init/systemd)
```

```
//ZOMBIE
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/types.h>
int main() {
   pid_t pid = fork();
   if (pid < 0) {
      printf("Fork failed!\n");
      exit(1);
   }else if (pid == 0) {
      printf("Child process started (PID: %d)\n", getpid());
      printf("Child exiting immediately...\n");
      exit(0);
   } else {
      printf("Parent process (PID: %d) sleeping...\n", getpid());
      sleep(10);
      printf("Parent finished sleeping. Now exiting.\n");
   }
   return 0;
}
```

OUTPUT (ZOMBIE PROCESS):



CODE 4:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Process {
int pid;
int at;
int bt;
int ct;
int tat;
int wt;
} Process;

void swap(Process *a, Process *b) {
Process t = *a;
*a = *b;
*b = t;
}

void sort(Process P[], int n) {
for(int i = 0; i < n - 1; i++) {
for(int j = i + 1; j < n; j++) {
if(P[i].at > P[j].at) {
swap(&P[i], &P[j]);
}
}
}
}

int main() {
int n;
printf("Enter number of processes: ");
scanf("%d", &n);
Process P[n];
for(int i = 0; i < n; i++) {
printf("Enter pid, AT, BT: ");
scanf("%d %d %d", &P[i].pid, &P[i].at, &P[i].bt);
}
sort(P, n);
int time = 0, totalTAT = 0, totalWT = 0;
for(int i = 0; i < n; i++) {
if(time < P[i].at)
time = P[i].at;
time += P[i].bt;
P[i].ct = time;
P[i].tat = P[i].ct - P[i].at;
P[i].wt  = P[i].tat - P[i].bt;
totalTAT += P[i].tat;
```

```
totalWT  += P[i].wt;
}
printf("\n%-5s %-5s %-5s %-5s %-5s %-5s\n",
    "pid", "AT", "BT", "CT", "TAT", "WT");
for(int i = 0; i < n; i++) {
printf("%-5d %-5d %-5d %-5d %-5d %-5d\n",
    P[i].pid, P[i].at, P[i].bt,
    P[i].ct, P[i].tat, P[i].wt);
}
printf("\nAverage TAT = %.2f\n", (float)totalTAT / n);
printf("Average WT  = %.2f\n", (float)totalWT / n);
return 0;
}
```

OUTPUT:

```
atharvmishra@MacBook-Air Operating System %  g++ fcfs.cpp
atharvmishra@MacBook-Air Operating System % ./a.out
Enter number of processes: 5
Enter pid, AT, BT: 1 0 4
Enter pid, AT, BT: 2 1 3
Enter pid, AT, BT: 3 2 1
Enter pid, AT, BT: 4 3 2
Enter pid, AT, BT: 5 4 5

pid   AT    BT    CT    TAT   WT
1     0     4     4     4     0
2     1     3     7     6     3
3     2     1     8     6     5
4     3     2     10    7     5
5     4     5     15    11    6

Average TAT = 6.80
Average WT  = 3.80
atharvmishra@MacBook-Air Operating System %
```

CODE 5:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Process {
   int pid;
   int at;
   int bt;
   int ct;
   int tat;
   int wt;
} Process;
void swap(Process *a, Process *b) {
   Process t = *a;
   *a = *b;
   *b = t;
}
void sortByArrival(Process P[], int n) {
   for(int i = 0; i < n - 1; i++) {
      for(int j = i + 1; j < n; j++) {
         if(P[i].at > P[j].at)
            swap(&P[i], &P[j]);
      }
   }
}
int main() {
   int n;
   printf("Enter number of processes: ");
   scanf("%d", &n);
   Process P[n];
   int visited[n];
   for(int i = 0; i < n; i++) {
      printf("Enter pid, AT, BT: ");
      scanf("%d %d %d", &P[i].pid, &P[i].at, &P[i].bt);
```

```
    visited[i] = 0;
}
sortByArrival(P, n);
int completed = 0, time = 0;
int totalTAT = 0, totalWT = 0;
while(completed < n) {
    int idx = -1;
    int minBT = 1e9;
    for(int i = 0; i < n; i++) {
        if(!visited[i] && P[i].at <= time) {
            if(P[i].bt < minBT) {
                minBT = P[i].bt;
                idx = i;
            }
        }
    }
    if(idx == -1) {
        time++;
        continue;
    }
    time += P[idx].bt;
    P[idx].ct = time;
    P[idx].tat = P[idx].ct - P[idx].at;
    P[idx].wt  = P[idx].tat - P[idx].bt;
    totalTAT += P[idx].tat;
    totalWT  += P[idx].wt;
    visited[idx] = 1;
    completed++;
}
printf("\n%-5s %-5s %-5s %-5s %-5s %-5s\n",
       "PID", "AT", "BT", "CT", "TAT", "WT");
for(int i = 0; i < n; i++) {
    printf("%-5d %-5d %-5d %-5d %-5d %-5d\n",
           P[i].pid, P[i].at, P[i].bt,
```

```
            P[i].ct, P[i].tat, P[i].wt);
    }
    printf("\nAverage TAT = %.2f\n", (float)totalTAT / n);
    printf("Average WT  = %.2f\n", (float)totalWT / n);
    return 0;
}
```

OUTPUT:

```
atharvmishra@MacBook-Air Operating System %  g++ sjf.cpp
atharvmishra@MacBook-Air Operating System % ./a.out
Enter number of processes: 5
Enter pid, AT, BT: 1 0 3
Enter pid, AT, BT: 2 1 5
Enter pid, AT, BT: 3 2 2
Enter pid, AT, BT: 4 9 1
Enter pid, AT, BT: 5 5 5

PID   AT    BT    CT    TAT   WT
1     0     3     3     3     0
2     1     5     10    9     4
3     2     2     5     3     1
5     5     5     16    11    6
4     9     1     11    2     1

Average TAT = 5.60
Average WT  = 2.40
atharvmishra@MacBook-Air Operating System %
```

CODE 6:

```
#include <stdio.h>
typedef struct {
   int pid, at, bt, pr;
   int ct, tat, wt;
} Process;

int main() {
   int n;
   printf("Enter number of processes: ");
   scanf("%d", &n);

   Process P[n];
   int visited[n];
   printf("Enter PID, AT, BT, Priority: ");
   for(int i = 0; i < n; i++) {
      scanf("%d %d %d %d", &P[i].pid, &P[i].at, &P[i].bt, &P[i].pr);
      visited[i] = 0;
   }

   int completed = 0, time = 0;
   int totalTAT = 0, totalWT = 0;

   while(completed < n) {
      int idx = -1;
      int bestPriority = 999999;
      for(int i = 0; i < n; i++) {
         if(!visited[i] && P[i].at <= time) {
            if(P[i].pr < bestPriority) {
               bestPriority = P[i].pr;
               idx = i;
            }
         }
      }
```

```
        if(idx == -1) {
            time++;
            continue;
        }
        time += P[idx].bt;
        P[idx].ct = time;
        P[idx].tat = P[idx].ct - P[idx].at;
        P[idx].wt  = P[idx].tat - P[idx].bt;
        totalTAT += P[idx].tat;
        totalWT  += P[idx].wt;
        visited[idx] = 1;
        completed++;
    }

    printf("\nPID  AT  BT  PR  CT  TAT  WT\n");
    for(int i = 0; i < n; i++) {
        printf("%d   %d   %d   %d   %d   %d   %d\n",
               P[i].pid, P[i].at, P[i].bt, P[i].pr,
               P[i].ct, P[i].tat, P[i].wt);
    }
    printf("\nAverage TAT = %.2f\n", (float)totalTAT / n);
    printf("Average WT  = %.2f\n", (float)totalWT / n);
    return 0;
}
```

OUTPUT:

```
atharvmishra@MacBook-Air Operating System %  g++ non_preem.cpp
atharvmishra@MacBook-Air Operating System % ./a.out
Enter number of processes: 5
Enter PID, AT, BT, Priority:
1 0 4 2
2 1 3 1
3 2 1 3
4 3 2 2
5 4 4 1

PID  AT  BT  PR  CT  TAT  WT
1    0   4   2   4   4    0
2    1   3   1   7   6    3
3    2   1   3   14   12    11
4    3   2   2   13   10    8
5    4   4   1   11   7    3

Average TAT = 7.80
Average WT  = 5.00
atharvmishra@MacBook-Air Operating System %
```

CODE 7:

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>

int main()
{
int fd[2];
pid_t p;
char re[100], wr[] = "Hi GEU Students";

if (pipe(fd) == -1)
{
printf("unable to create pipe\n");
return 1;
}

p = fork();
if (p > 0)
{
close(fd[0]);
printf("Parent writing message to pipe\n");
write(fd[1], wr, strlen(wr) + 1);
close(fd[1]); // Done writing
printf("Parent completed the writing\n");
}
else if(p==0)
{
close(fd[1]);
printf("Child reading message from pipe\n");
read(fd[0], re, sizeof(re));
printf("Child received: %s\n", re);
close(fd[0]);
}
```

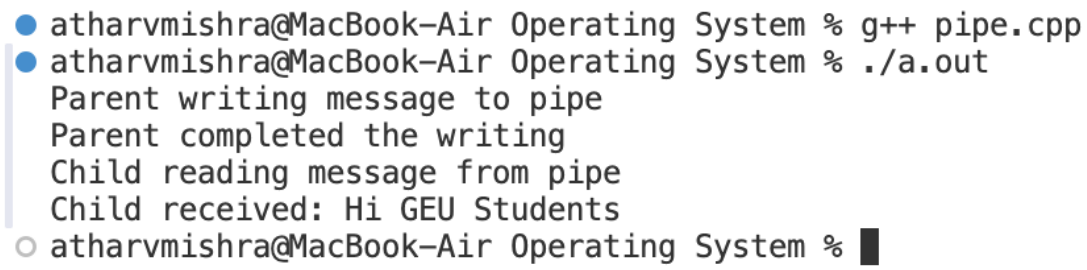
```
else

perror("No child created\n");

return 0;

}
```

OUTPUT:



CODE 8:

```
#include <stdio.h>
#include <string.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

int main()
{
int fd;
char * myfifo = "/tmp/myfifo";
mkfifo(myfifo, 0666);
char arr1[80], arr2[80];
while (1)
{
fd = open(myfifo, O_WRONLY);
fgets(arr2, 80, stdin);
write(fd, arr2, strlen(arr2)+1);
close(fd);
fd = open(myfifo, O_RDONLY);
read(fd, arr1, sizeof(arr1));
printf("User2: %s\n", arr1);
close(fd);
}
return 0;
}
```

```
#include <stdio.h>
#include <string.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

int main()
{
int fd1;
char * myfifo = "/tmp/myfifo";
mkfifo(myfifo, 0666);
char str1[80], str2[80];
while (1)
{
fd1 = open(myfifo, O_RDONLY);
read(fd1, str1, 80);
printf("User1: %s\n", str1);
close(fd1);
fd1 = open(myfifo, O_WRONLY);
fgets(str2, 80, stdin);
write(fd1, str2, strlen(str2)+1);
close(fd1);
}
return 0;
}
```

OUTPUT:

```
atharvmishra@MacBook-Air Operating System % g++ fifo1.cpp
atharvmishra@MacBook-Air Operating System % ./a.out
HELLO THERE!
User2: HELLO
```

```
atharvmishra@MacBook-Air Operating System % g++ fifo2.cpp
atharvmishra@MacBook-Air Operating System % ./a.out
User1: HELLO THERE!

HELLO
```

CODE 9:

```
#include <stdio.h>
int main() {
   int frames, pages, i, j, k, faults = 0;
   int frameQueue[10], page[30];
   printf("Enter number of frames: ");
   scanf("%d", &frames);
   printf("Enter number of pages: ");
   scanf("%d", &pages);
   printf("Enter page reference string:\n");
   for (i = 0; i < pages; i++)
      scanf("%d", &page[i]);
   for (i = 0; i < frames; i++)
      frameQueue[i] = -1;
   k = 0;
   for (i = 0; i < pages; i++) {
      int flag = 0;
      for (j = 0; j < frames; j++)
         if (frameQueue[j] == page[i])
            flag = 1;
      if (flag == 0) {
         frameQueue[k] = page[i];
         k = (k + 1) % frames;
         faults++;
      }
      printf("\nFrames: ");
      for (j = 0; j < frames; j++)
         printf("%d ", frameQueue[j]);
   }
   printf("\nTotal page faults = %d\n", faults);
   return 0;
}
```

OUTPUT:

```
atharvmishra@MacBook-Air Operating System % g++ fifo_page.cpp
atharvmishra@MacBook-Air Operating System % ./a.out
Enter number of frames: 3
Enter number of pages: 10
Enter page reference string: 1 2 3 2 4 1 5 2 4 3

Total Page Faults: 9
Total Page Hits: 1
Page Fault Rate: 0.9
atharvmishra@MacBook-Air Operating System %
```

CODE 10:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
   int n, head, i, total = 0;
   printf("Enter number of disk requests: ");
   scanf("%d", &n);
   int req[n];
   printf("Enter disk request sequence:\n");
   for (i = 0; i < n; i++)
      scanf("%d", &req[i]);
   printf("Enter initial head position: ");
   scanf("%d", &head);
   for (i = 0; i < n; i++) {
      total += abs(req[i] - head);
      head = req[i];
   }
   printf("Total Head Movement = %d\n", total);
   return 0;
}
```

OUTPUT:

```
atharvmishra@MacBook-Air Operating System % g++ fcfs_disk.cpp
atharvmishra@MacBook-Air Operating System % ./a.out
Enter number of disk requests: 7
Enter disk request sequence:
80 170 43 140 24 16 190
Enter initial head position: 30
Total Head Movement = 662
atharvmishra@MacBook-Air Operating System %
```